

# Rapport final — Synthèse pour passage à la construction

Suite Osiris — PWA multi-outils enseignant

## 1. Décision de fond

**PWA : validée.** Installation cross-plateforme sans store, coquille + service worker s'ajoutent à l'architecture existante sans la remettre en cause, et le hors-ligne cohabite naturellement avec les fonctionnalités en ligne (dictionnaire, futur trieur IA) — le principe est déjà appliqué dans la suite elle-même (bascule En ligne/Hors ligne du Wortfamilie-Finder, chargement sql.js local-first avec repli CDN). Réserve à garder en tête : Safari/iOS plus restrictif sur les service workers.

## 2. Périmètre final de la PWA

| App                                        | Rôle                                                                | Statut                                    |
|--------------------------------------------|---------------------------------------------------------------------|-------------------------------------------|
| PORTFOLIO_SEITE.html                       | Coquille (onglets, rail, réglages)                                  | Existant, stable                          |
| DASHBOARD.html                             | Tableau de bord enseignant                                          | Existant, à intégrer                      |
| KI_WÖRTERBUCH.html                         | Dictionnaire principal                                              | Existant, à intégrer                      |
| KI_SPEZIALISIERTES_WÖRTERBUCH.html         | Dictionnaire spécialisé                                             | Existant, à intégrer                      |
| MEINE_BANK1_5_4_2.html                     | Gestion documentaire                                                | Existant, à intégrer                      |
| MEIN_OFFLINE_WIKI_1_2.html                 | Wiki hors-ligne                                                     | <b>Nettoyage requis avant intégration</b> |
| BIBLIOTHEK.html ( <i>nom provisoire</i> )  | Import de fichiers, extracteur de texte, futur trieur IA par niveau | À construire                              |
| Générateur de tests IA, lecteur de musique | Modules optionnels                                                  | Différés — hors périmètre du socle        |

Le socle non retirable reste le Dashboard ; dictionnaire, bibliothèque de fichiers et wiki sont des défauts recommandés mais fermables, ajoutables/retirables sans jamais perturber les autres onglets ouverts (mécanisme déjà en place via `userLinks`).

## 3. Structure de dossiers définitive

```
/
PORTFOLIO_SEITE.html
DASHBOARD.html
KI_WÖRTERBUCH.html
KI_SPEZIALISIERTES_WÖRTERBUCH.html
MEINE_BANK1_5_4_2.html
MEIN_OFFLINE_WIKI_1_2.html
BIBLIOTHEK.html
manifest.json
sw.js

/lib/
/chart/v4.5.1/ → chart.umd.min.js [Dashboard]
/sqljs/v1.10.3/ → sql-wasm.js, sql-wasm.wasm [2 dictionnaires - deja partage sans modif de code]
```

```

/mammoth/vX/ → mammoth.browser.js [Meine Bank, Bibliothek]
/pdf/vX/ → pdf.min.js, pdf.worker.min.js, pdf-lib.min.js [Meine Bank, Bibliothek]
/jszip/vX/ → jszip.min.js [Meine Bank, Portfolio, Bibliothek]
/htmldocx/vX/ → html-docx.js [Offline Wiki uniquement]

/icons/ (jeu fixe par défaut, un par app + upload custom)
/fonts/ (4 polices actuelles + slot custom upload)

```

Convention retenue : chaque sous-dossier de `/lib/` est nommé par bibliothèque **et versionné** — un changement de version = nouveau dossier = nouveau nom de cache côté service worker, qui purge l'ancien à l'activation. Le service worker ne précache que les bibliothèques réellement utilisées par les apps installées chez l'utilisateur, jamais tout `/lib/` par défaut.

## 4. Actions classées par impact / effort

| # | Action                                                                                                     | Impact                                                 | Effort                   | Priorité        |
|---|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|--------------------------|-----------------|
| 1 | Extraire html-docx.js hors de MEIN_OFFLINE_WIKI vers /lib/htmldocx/, corriger le <script src> déjà présent | Fichier ÷14 (471 Ko → ~33 Ko)                          | Faible — extraction pure | Très haute      |
| 2 | Créer /lib/sqljs/v1.10.3/ et y placer sql-wasm.js/.wasm                                                    | Débloque les 2 dictionnaires sans toucher à leur code  | Très faible              | Haute           |
| 3 | Déplacer chart.js, mammoth, pdf.js/pdf-lib, jszip vers dossiers versionnés + corriger les chemins          | Mutualisation réelle entre apps                        | Faible-moyen             | Haute           |
| 4 | Corriger la troncature du bouton de fermeture du dernier onglet                                            | Bug visible, gênant à l'usage                          | Faible                   | Moyenne-haute   |
| 5 | Manifest + service worker avec précache sélectif                                                           | Rend l'app installable/hors-ligne                      | Moyen                    | Moyenne         |
| 6 | Construire BIBLIOTHEK.html (import + extraction de texte)                                                  | Débloque wiki-par-import et futur générateur de tests  | Moyen-élevé              | Moyenne         |
| 7 | Trieur IA par niveau (dans Bibliothek)                                                                     | Appel réseau direct, pas de dépendance locale nouvelle | Faible, différable       | Basse — roadmap |
| 8 | Icônes/polices par défaut en tableaux fixes                                                                | Cohérence visuelle                                     | Faible                   | Basse           |

## 5. Spécification resserrée de BIBLIOTHEK.html

- **Import de fichiers** : PDF/Word en priorité, en réutilisant tel quel le trio déjà fonctionnel de Meine Bank (mammoth.browser.js, pdf.min.js+worker, pdf-lib.min.js) — aucune nouvelle bibliothèque à sourcer.
- **Extracteur de texte** : sortie destinée à deux consommateurs identifiés — l'import « Text Content » du wiki (aujourd'hui limité au JSON/saisie manuelle) et, plus tard, le générateur de tests IA.
- **Trieur par niveau IA** : appel réseau direct vers un fournisseur IA (même schéma que le Wortfamilie-Finder), à traiter comme un module « en ligne uniquement », sans urgence sur le plan des dépendances.
- Aucune fonctionnalité de lecture PDF/Word n'est à ajouter dans MEIN\_OFFLINE\_WIKI lui-même — confirmé : le wiki reste un simple récepteur de texte déjà extrait.

## 6. Ordre de construction recommandé

- 1 Nettoyage MEIN\_OFFLINE\_WIKI (action #1) — gain immédiat, zéro risque, rentable en une session.
- 2 Mise en place de /lib/sqljs/ (action #2) — débloque les deux dictionnaires en quelques minutes.
- 3 Correctif du bouton de fermeture d'onglet (action #4) — petit bug visible, bon gain de confiance rapide.
- 4 Migration des chemins des autres bibliothèques vers /lib/ versionné (action #3).
- 5 Manifest + service worker minimal (action #5) — à ce stade, la coquille + les 5 apps existantes sont déjà installables et partiellement hors-ligne.
- 6 Construction de BIBLIOTHEK.html (action #6) — première version : import + extraction de texte vers le wiki, sans encore le trieur IA.
- 7 Trieur IA par niveau, puis modules optionnels (générateur de tests, lecteur de musique) une fois le socle stable.

Cet ordre priorise ce qui est **immédiatement livrable et à faible risque** (nettoyage, dossier partagé, petit bug UI) avant les chantiers plus structurants (PWA, nouvelle app), pour obtenir des résultats visibles dès les premières sessions plutôt que de tout faire dépendre de la nouvelle app.